

iLOcate

A commute and travel guide for Iloilo City

Member	Role
Reinwel G. Tingson (@Reeiinn)	Developer: UI, state management, API and data layer, QA and testing, documentation, Git and repository manager

SE2142 · Software Engineering Tools and Practices

Final Project Documentation Paper

Section: _____

Date of submission: _____

Repository: **https://github.com/Reeiinn/iLOcate**

Contents

1. Member Roles
2. Problem Statement
3. Application Overview
4. Proposed Solution
5. Goals and Objectives
6. Features
7. Technical Implementation
    1. Navigation
    2. State Management
    3. API Integration
    4. CRUD and Local Storage
    5. Error and Edge-Case Handling
    6. Widget Composition and Code Organization
8. Software Engineering Tools and Practices
    1. Git, GitHub and Branching
    2. Issue Tracking
    3. Static Analysis and Testing
    4. Debugging Case
    5. Release Preparation
9. References

1. Member Roles

I completed this project individually, so every role listed in the project brief is assigned to me. The table below breaks the work into those roles so it is clear which parts of the system each one covers.

Member	Role	Responsibilities
Reinwel G. Tingson	UI lead	Screen layouts, Material 3 theme (light and dark), reusable widgets in `lib/widgets/`
	State management	Provider setup in `main.dart`; `PlacesProvider`, `LikesProvider`, `TripsProvider`, `SettingsProvider`, `LocationProvider`
	API / data layer	`ApiClient`, `WeatherService` (Open-Meteo), `DirectionsService` (OSRM), model classes, `LocalStore`
	QA / testing	22 unit and widget tests in `test/`, `flutter analyze`, manual testing of every screen online and offline
	Documentation	README, code comments, this documentation paper
	Git / repository manager	GitHub issues #1 to #5, one feature branch per feature, commit history, pull requests

2. Problem Statement

Iloilo City is served mainly by jeepneys (PUJs) that run fixed routes, but the routes are not posted at most stops and there is no official map that riders can check on a phone. A student, a new resident or a visitor who wants to reach a church in Molo or a batchoy place in La Paz usually has to ask a driver or a stranger which jeepney to take. People who are new to the city often guess, ride the wrong route, or fall back on a taxi or a ride-hailing car, which costs several times the jeepney fare.

The same visitors also want to know what is worth seeing. Information about local cafes, heritage houses and churches is spread across social media posts and travel blogs, and none of it is tied to how you would actually get there by public transport. Weather adds another layer: afternoon rain is common, and a 15-minute walk from the jeepney stop to a place is a different trip in a downpour.

Current options do not close this gap. General map apps give car or walking directions but have little or no data on Iloilo's jeepney routes. Route maps shared on Facebook are images that cannot be searched or combined with a destination. Planning a day out therefore means switching between several apps and asking people for help.

Target users: college students who commute around the city, new residents and workers, and local or foreign tourists who want to visit Iloilo's food spots and heritage sites using public transport.

3. Application Overview

iLOcate is an Android app, written in Flutter and Dart, that puts Iloilo City's jeepney routes and 98 hand-picked places (restaurants, cafes, churches, museums, heritage houses, malls and beaches) on one interactive map. Users can browse and search places, see the current weather at a place, get road directions between two points, see which jeepney routes pass near both ends of the trip, and save places into trip plans that stay on the phone.

The app opens on the map tab. A bottom navigation bar switches between four tabs: **Map**, **Places**, **Trips** and **Likes**. Tapping a place anywhere in the app opens its detail screen, which is the hub for the weather card, directions and adding the place to a trip. Place and route data are bundled with the app, so the lists and routes still work without a connection. Map tiles, weather and directions come from online services and show a clear message with a Retry button when the phone is offline.

> **Source code repository:** https://github.com/Reeiinn/iLOcate (public)

4. Proposed Solution

iLOcate brings the three things a commuter needs, namely where a place is, how to get there and what the conditions are, into one app. The typical workflow is:

1. The user opens the app and sees the city map with color-coded pins. They can switch on any of the 27 jeepney routes from the **Routes** button, or press the GPS button to see where they are.
2. They find a destination on the map or in the **Places** tab, using the search box and category chips.
3. The place's detail screen shows a photo, its category, the current weather and a small map.
4. **Directions** draws the road route from a starting point (another place or the user's GPS position), shows the distance and travel time, and lists the jeepney routes that pass within 500 m of both ends.
5. **Add to trip** saves the place into a new or existing trip plan. Trips have a name, date, notes and an ordered list of stops, and each stop can show directions from the stop before it.

The technical choices follow from this workflow. The list of places, the liked places, the trips, the theme and the GPS position are each needed on several screens, so they live in **Provider** `ChangeNotifier` classes instead of being passed down through constructors. Weather and routing change all the time and cannot be bundled, so they come from two free **REST APIs** (Open-Meteo and OSRM) called with the `http` package. The jeepney-matching logic runs locally on the bundled route data, which keeps it fast and available offline. Trips and likes are saved with `shared_preferences` so they survive an app restart without needing an account or a server.

5. Goals and Objectives

The overall goal is a working prototype that makes it easier to plan a trip around Iloilo City by jeepney. The specific objectives were:

1. Show all 27 jeepney routes and 98 places on one map that a user can filter and explore, and that still loads its place and route data without an internet connection.
2. Integrate two REST APIs through the `http` package (Open-Meteo for weather, OSRM for road directions), parse their JSON into Dart model classes, and show loading, success and error states for each.
3. Suggest jeepney routes for a trip by finding routes that pass within walking distance (500 m) of both the start and the destination.
4. Give the user full create, read, update and delete control over trip plans, with form validation, and keep trips and likes on the device between sessions.
5. Follow the software engineering practices from the course: a layered folder structure, Provider for shared state, named routes, GitHub issues and feature branches, a clean `flutter analyze`, automated tests, a documented debugging case, and a release build with a custom icon and splash screen.

6. Features

All screenshots below were taken from the running app, at phone size (412 × 915 logical pixels), with two sample trips and four liked places created for the demonstration. Weather values and routes are live responses from the APIs at the time of capture.

Feature	Type	Course requirement shown
Interactive map with 98 color-coded place pins and a place preview sheet	Core	Widgets, packages, navigation
Jeepney route picker that draws any of the 27 PUJ routes	Core	Stateful widgets, bottom sheets
GPS "my location" button and blue position dot	Supporting	Shared state (LocationProvider), permissions
Places & Food list with search and category filters	Core	Data passed to detail screen, empty state
Place details with live weather card	Core	REST API, FutureBuilder, error/retry
Directions with OSRM route, distance, time and matching jeepneys	Core	REST API, JSON model, local algorithm
Trips: create, view, edit, delete (swipe with undo), add places	Core	CRUD, form validation, Provider
Likes saved on the device, with a count badge on the tab	Supporting	Provider, shared_preferences
Settings: light, dark or system theme; clear likes; about	Supporting	App-wide state, theming
Offline and server error messages, empty states, not-found route	Supporting	Error and edge-case handling

6.1 Map and jeepney routes

The Map tab is the home screen. Pins are colored by category. The Routes button opens a checklist of all 27 jeepney routes; checked routes are drawn on the map in their own colors, in both directions.

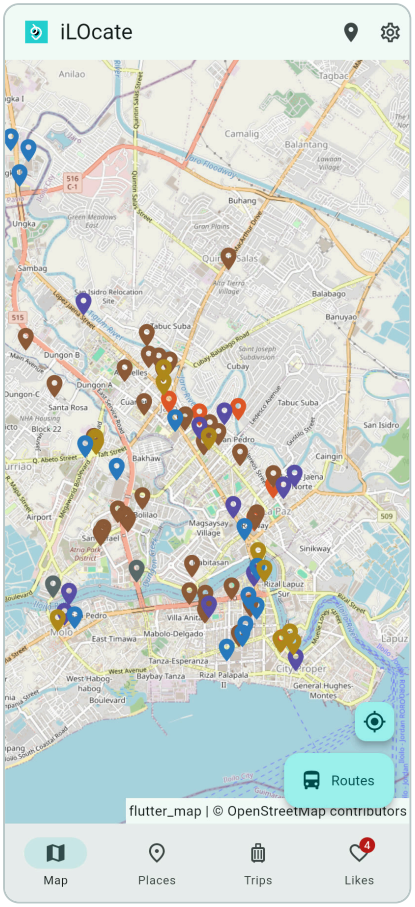


Figure 1. Map tab with all place pins.

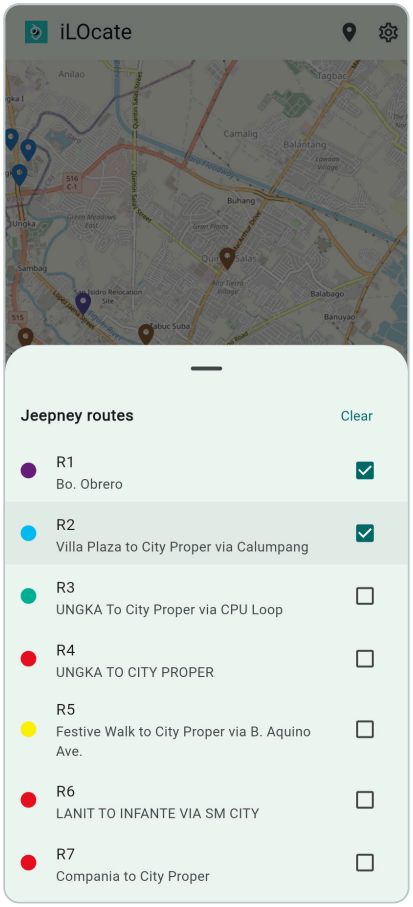


Figure 2. Jeepney route picker with R1 and R2 selected.

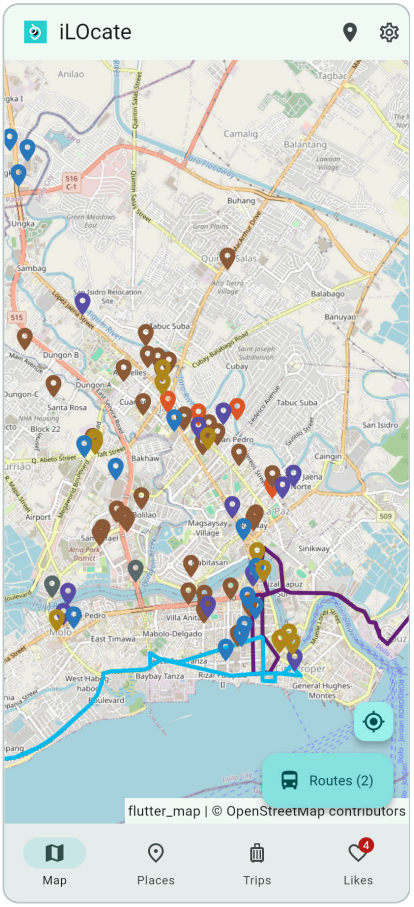


Figure 3. Selected routes drawn on the map.

6.2 Places and Food

The Places tab lists every place with its photo and category. The search box and category chips filter the list as the user types. When nothing matches, an empty state explains why and offers to clear the filters.

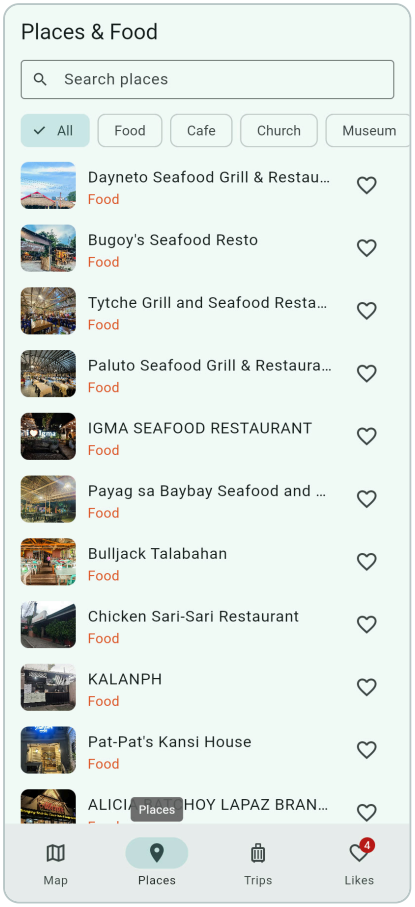


Figure 4. Places & Food list.

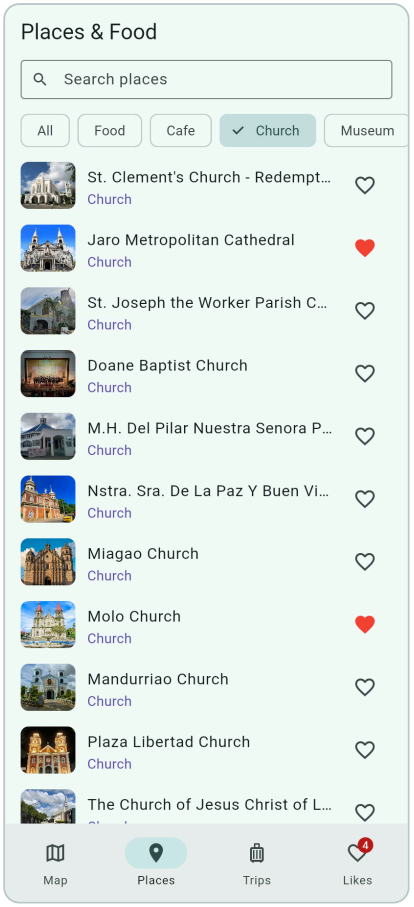


Figure 5. Filtered to churches.

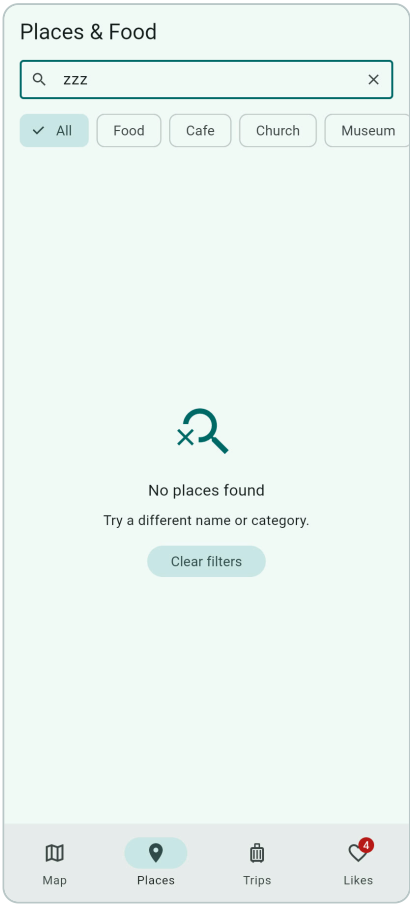


Figure 6. Empty state for a search with no results.

6.3 Place details, weather and add to trip

Opening a place shows its photo, category chip, coordinates, a live weather card from Open-Meteo, and a small map. The heart in the app bar likes the place. **Add to trip** opens a sheet that lists existing trips or starts a new trip with this place already selected.

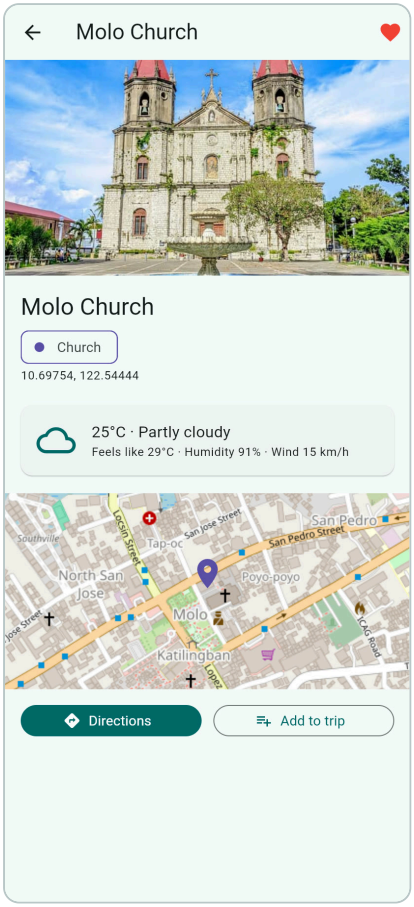


Figure 7. Place details with live weather.

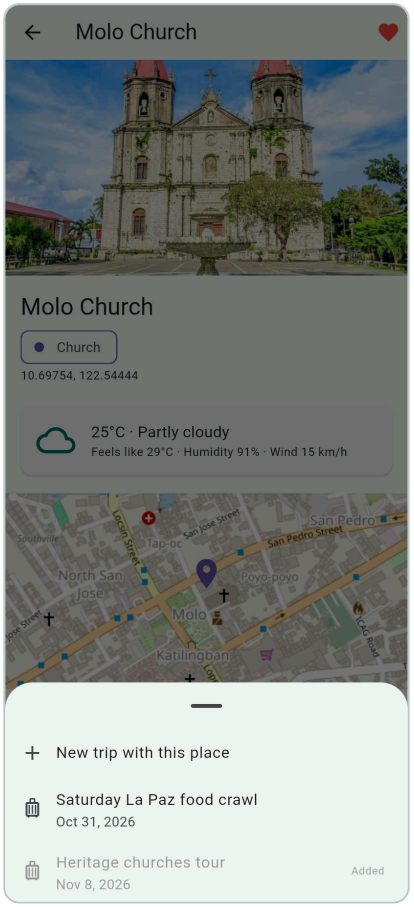


Figure 8. Add-to-trip sheet.

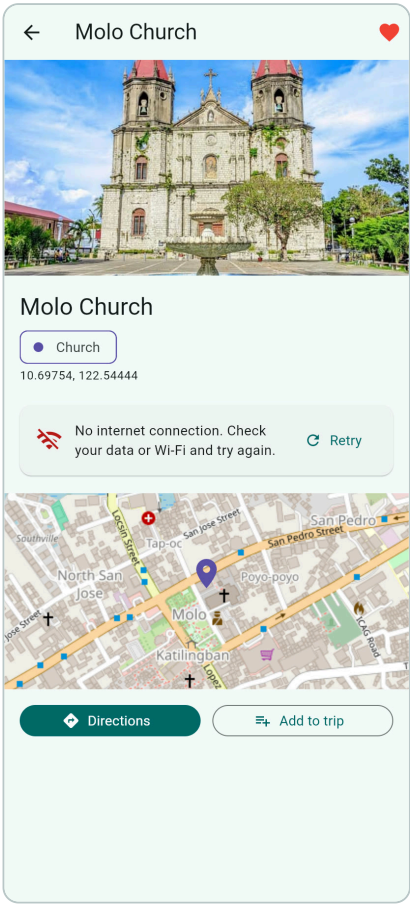


Figure 9. Weather card when the phone is offline, with Retry.

6.4 Directions

The Directions screen has searchable From and To fields. The start defaults to the landmark nearest the city center, and can be changed to any place or to *My location* (GPS). The OSRM route is drawn on the map with its distance and estimated driving time, followed by the jeepney routes that pass near both ends.

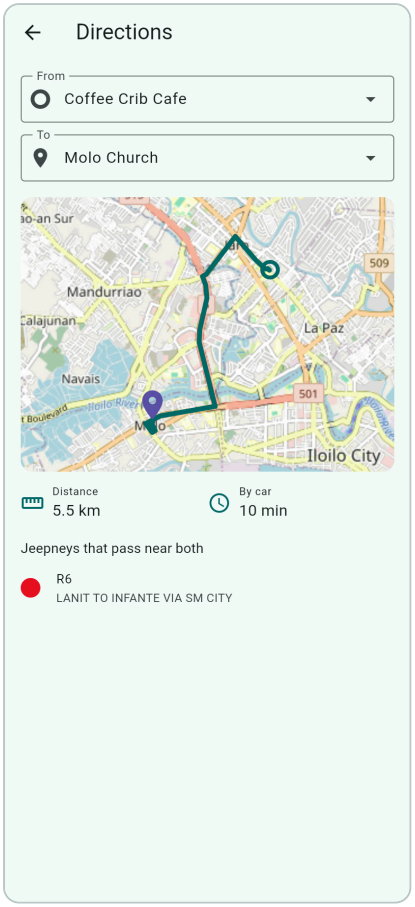


Figure 10. Directions from a cafe to Molo Church with the matching jeepney route.

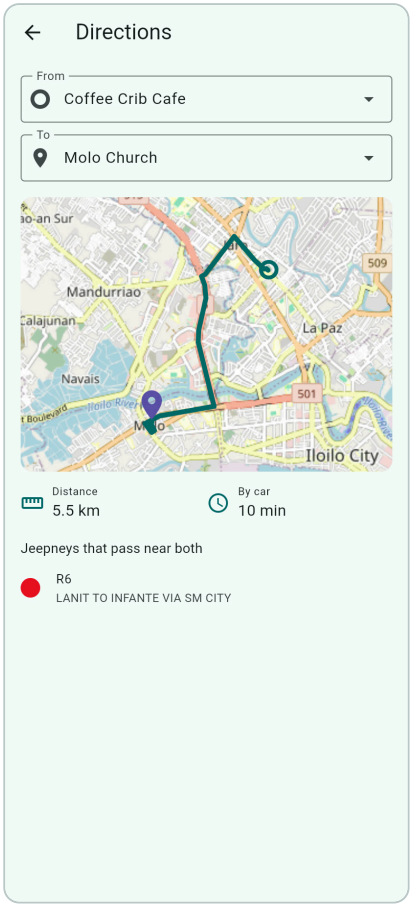


Figure 11. Directions screen scrolled to the jeepney list.

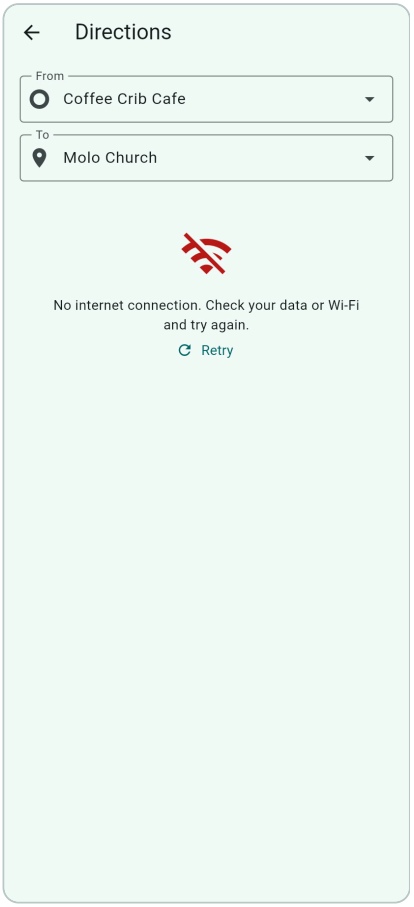


Figure 12. Directions when offline: message and Retry.

6.5 Trips (CRUD)

The Trips tab lists saved trips by date. Each card shows the date, name, number of stops and photos. Tapping a card opens the trip details with its notes and stops; every stop after the first has a button for directions from the previous stop. The pencil icon edits the trip and the trash icon deletes it after a confirmation. On the list, swiping a trip to the left deletes it with an Undo option.

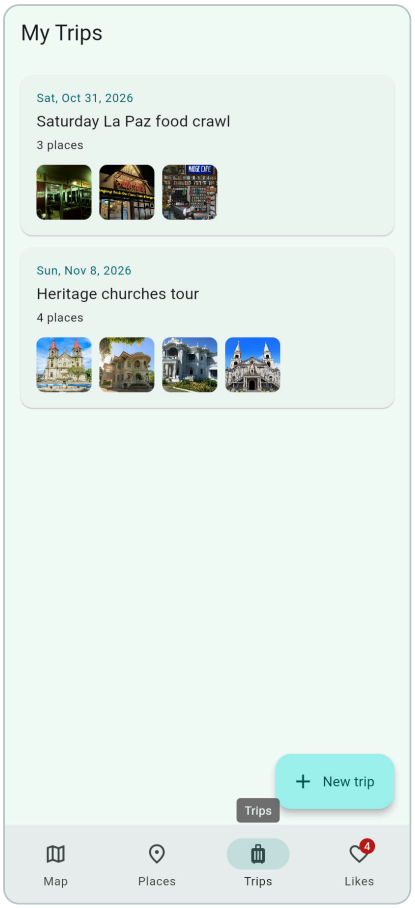


Figure 13. Trips list (Read).

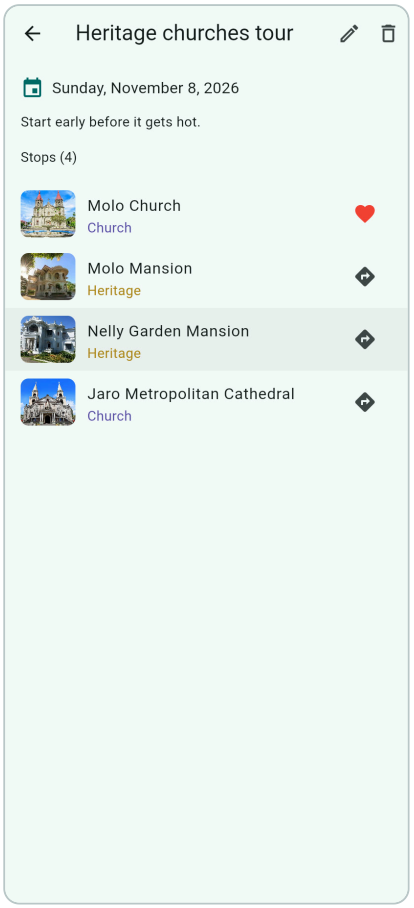


Figure 14. Trip details with stops.

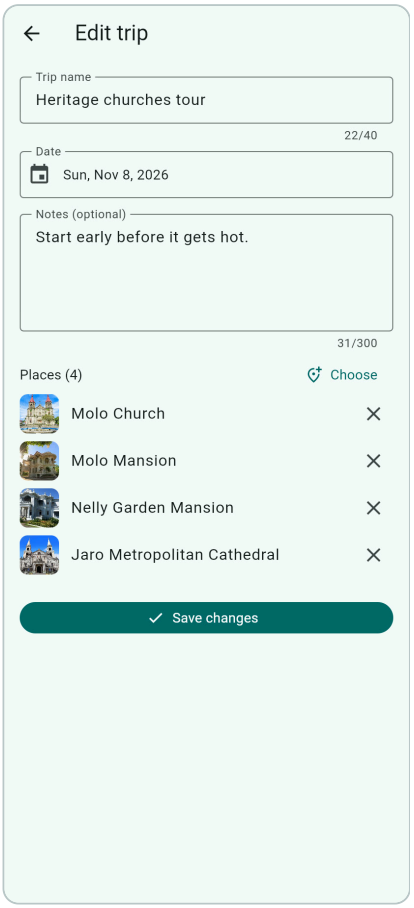


Figure 15. Edit trip form (Update).

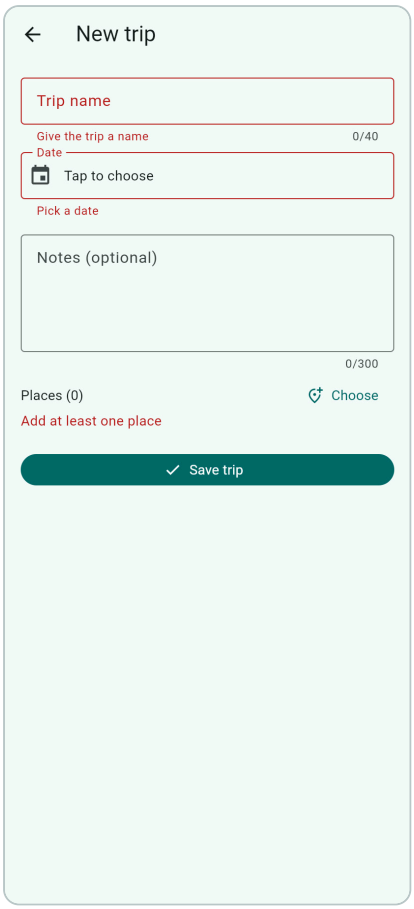


Figure 16. New trip form (Create) showing validation errors.

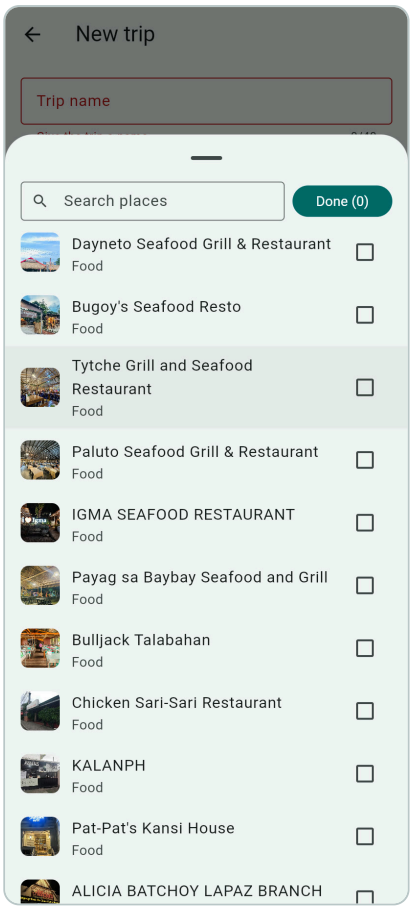


Figure 17. Searchable place picker used by the form.

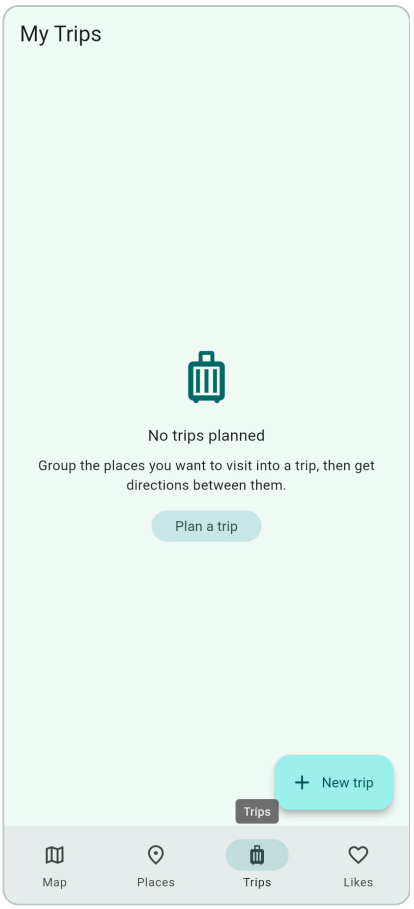


Figure 18. Trips empty state before any trip exists.

6.6 Likes, settings and theming

Liked places appear in the Likes tab, and the tab icon shows a badge with the count. Settings switches between light, dark and system theme; the choice is saved and applied to the whole app immediately.

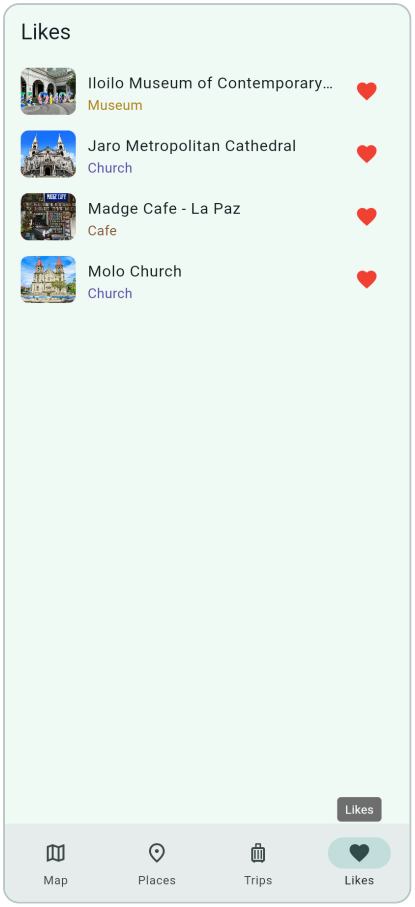


Figure 19. Likes tab.

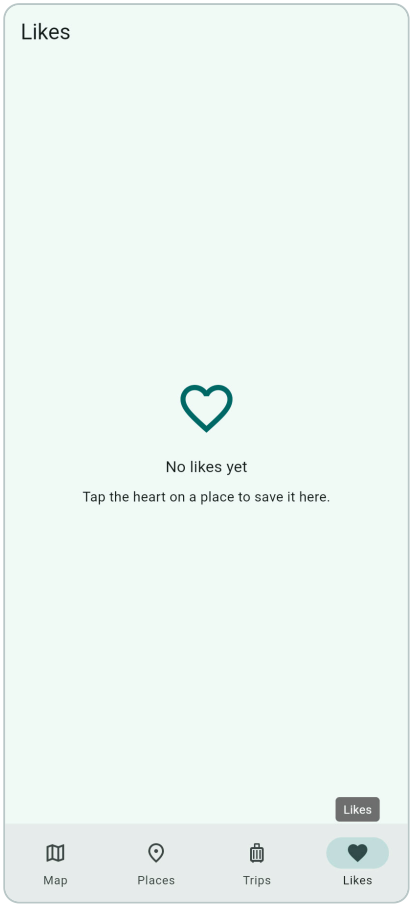


Figure 20. Likes empty state.

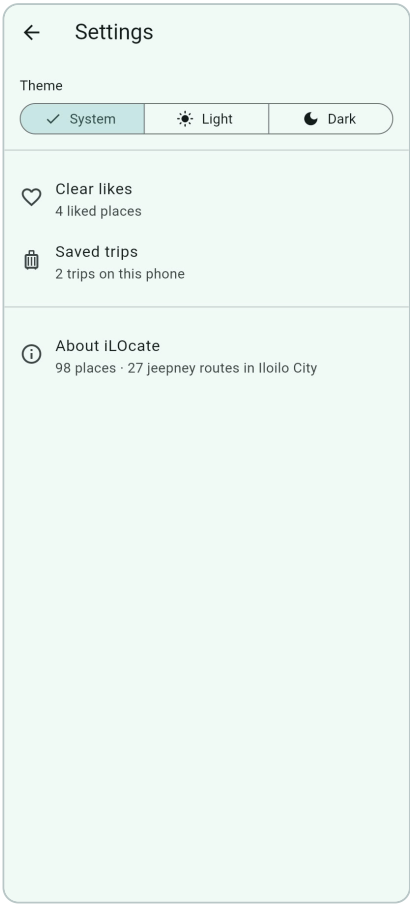


Figure 21. Settings screen.

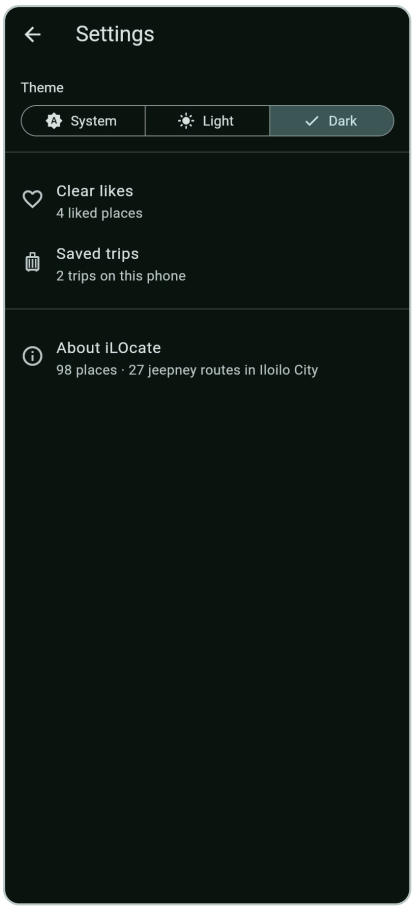


Figure 22. Settings after choosing Dark.

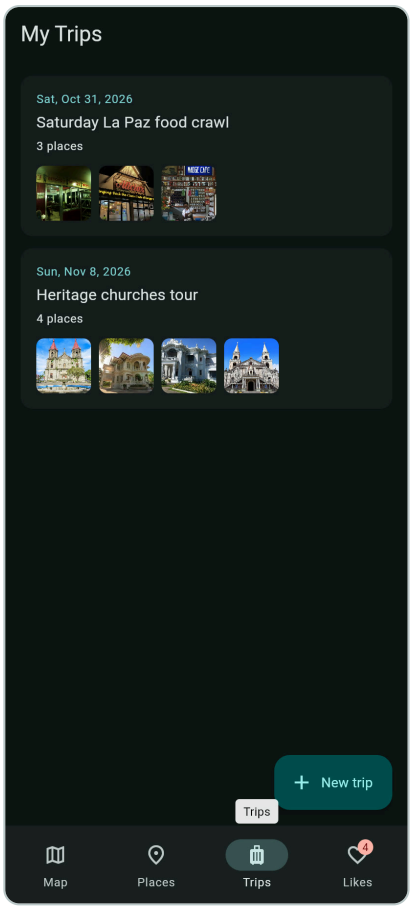


Figure 23. Trips tab in dark theme.



Figure 24. Custom launcher icon generated from the iLOcate logo. The splash screen shows the same logo on white (dark grey in dark mode).

7. Technical Implementation

This section explains, for each required area, what the app needed, how I built it (with the files, classes and endpoints involved), and what would be harder or would break without it.

7.1 Navigation

What the app needed

The app has eleven screens. Four of them are top-level tabs that the user switches between constantly, and the rest are pushed on top: place details, directions, trip details, the trip form, settings and a not-found page. Several screens need data from the screen that opened them: the detail screen needs the `Place` that was tapped, directions needs a destination (and optionally a start), and the trip form needs to know whether it is creating a new trip or editing an existing one. The same detail screen is opened from five places (map preview, places list, likes list, trip details and the trip card), so the way to open it must be the same everywhere.

How it was implemented

I used **named routes with `onGenerateRoute`** (Navigator 1.0). All route names live in one class, `AppRoutes`, in `lib/routes/app_router.dart`. `MaterialApp` in `lib/app.dart` sets `initialRoute: AppRoutes.home` and hands every route to the `onGenerateRoute` function. That function uses a Dart `switch` with `when` guards, so a route is only built if its argument has the right type; anything else falls through to `NotFoundScreen` instead of crashing.

lib/routes/app_router.dart

```dart
/// Every named route in the app. Screens navigate with
/// `Navigator.pushNamed(context, AppRoutes.x, arguments: ...)`.
abstract final class AppRoutes {
  static const home = '/';
  static const placeDetails = '/place';
  static const directions = '/directions';
  static const tripDetails = '/trip';
  static const tripForm = '/trip/form';
  static const settings = '/settings';
}
```

lib/routes/app_router.dart — builds a screen per route and type-checks its argument

```
Route<dynamic> onGenerateRoute(RouteSettings settings) {
  final args = settings.arguments;
  final Widget page = switch (settings.name) {
    AppRoutes.home ⇒ const HomeShell(),
    AppRoutes.placeDetails when args is Place ⇒
      PlaceDetailsScreen(place: args),
    AppRoutes.directions when args is Place ⇒
      DirectionsScreen(destination: args),
    AppRoutes.directions when args is DirectionsArgs ⇒
      DirectionsScreen(destination: args.to, start: args.from),
    AppRoutes.tripDetails when args is String ⇒
      TripDetailsScreen(tripId: args),
    AppRoutes.tripForm ⇒ TripFormScreen(
        args: args is TripFormArgs ? args : const TripFormArgs(),
      ),
    AppRoutes.settings ⇒ const SettingsScreen(),
    _ ⇒ NotFoundScreen(routeName: settings.name),
  };
  return MaterialPageRoute(builder: (_) ⇒ page, settings: settings);
}
```

Route name	Screen (file)	Argument passed	Opened from
`/`	`HomeShell` (home_shell.dart)	none	App start
`/place`	`PlaceDetailsScreen`	`Place`	Map preview, Places, Likes, Trip details
`/directions`	`DirectionsScreen`	`Place` or `DirectionsArgs(to, from)`	Place details, Trip details
`/trip`	`TripDetailsScreen`	`String` trip id	Trips tab
`/trip/form`	`TripFormScreen`	`TripFormArgs(trip?, initialPlaceId?)`	Trips tab, Trip details, Add to trip
`/settings`	`SettingsScreen`	none	Map app bar
anything else	`NotFoundScreen`	route name	Wrong name or argument

Data is passed with the `arguments` parameter of `Navigator.pushNamed`. For example, `PlaceTile` (a widget reused on three screens) opens the detail screen like this:

lib/widgets/place_tile.dart

```
      onTap: () ⇒ Navigator.pushNamed(
        context,
        AppRoutes.placeDetails,
        arguments: place,
      ),
```

Two decisions are worth explaining. First, the trip details route receives the trip's **id**, not the `Trip` object. The screen looks the trip up in `TripsProvider` every time it builds, so after the user

edits the trip and comes back, the details already show the new values. If I had passed the object, the screen would keep showing the old copy. Second, the trip form uses one route for both create and edit: an empty `TripFormArgs` means "new trip", and `TripFormArgs(trip: t)` means "edit t".

The four tabs are not routes. `HomeShell` (`lib/screens/home_shell.dart`) keeps a tab index and shows the tabs in an `IndexedStack` under a `NavigationBar`. An `IndexedStack` keeps every tab alive, so the map keeps its zoom and position and the search box keeps its text when the user switches tabs.

What would be harder without it

Without named routes, each of the five places that open the detail screen would build `MaterialPageRoute(builder: (_) ⇒ PlaceDetailsScreen(...))` itself and import the screen directly. Changing that screen's constructor would mean editing five files, and nothing would check that the right data was passed. With `onGenerateRoute` there is one map of the app and one place where arguments are checked, and a mistake leads to a "Page not found" screen instead of a red error screen.

7.2 State Management

What the app needed

Five pieces of state are used by more than one screen. The place list is needed by all four tabs and by the details, directions, trip and settings screens. The set of liked places is changed from the place tile, the detail screen and settings, and is shown in the Likes tab and in the badge on the tab bar. Trips are changed in the form, the list (swipe to delete), the details and the add-to-trip sheet. The theme is chosen in settings but applied by `MaterialApp` at the top of the tree. The GPS position is read by both the map and the directions screen. When one screen changes any of these, every other screen that shows it has to update.

How it was implemented

Each piece of shared state is a `ChangeNotifier` in `lib/providers/`, registered once in `lib/main.dart` with `MultiProvider`. The two API services are registered too, as plain `Provider`s, so widgets can get them with `context.read` and tests can swap them out.

lib/main.dart

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final store = LocalStore(await SharedPreferences.getInstance());
  final api = ApiClient(http.Client());

  runApp(
    MultiProvider(
      providers: [
        // Services: plain objects, read with context.read<T>().
        Provider.value(value: WeatherService(api)),
        Provider.value(value: DirectionsService(api)),
        // Shared app state: ChangeNotifiers that screens watch.
        ChangeNotifierProvider(
          create: (_) => PlacesProvider(AssetDataService())..load(),
        ),
        ChangeNotifierProvider(create: (_) => LikesProvider(store)),
        ChangeNotifierProvider(create: (_) => TripsProvider(store)),
        ChangeNotifierProvider(create: (_) => SettingsProvider(store)),
        ChangeNotifierProvider(
          create: (_) => LocationProvider(LocationService()),
        ),
      ],
      child: const ILocateApp(),
    ),
  );
}
```

Provider class (file)	State it holds	Main methods	Saved on device
`PlacesProvider` (places_provider.dart)	places, jeepney routes, load status	`load()`, `search()`, `placeById()`	No (bundled assets)
`LikesProvider` (likes_provider.dart)	set of liked place ids	`toggle()`, `isLiked()`, `clear()`	Yes
`TripsProvider` (trips_provider.dart)	list of trips, sorted by date	`add()`, `update()`, `delete()`, `restore()`, `addPlace()`	Yes
`SettingsProvider` (settings_provider.dart)	theme mode	`setThemeMode()`	Yes
`LocationProvider` (location_provider.dart)	last GPS position, locating flag, error	`locate()`	No

Widgets use three ways to reach a provider, depending on what they need:

Access	Meaning	Where it is used
`context.watch<T>()`	Rebuild this widget whenever T changes	`HomeShell`, `MapScreen`, `PlacesScreen`, `LikesScreen`, `TripsScreen`, `DirectionsScreen`, `SettingsScreen`
`context.select<T, R>()`	Rebuild only when one value inside T changes	`ILocateApp` (theme mode), `LikeButton` (is this place liked), `HomeShell` (like count badge), `TripDetailsScreen` (this trip), `SettingsScreen` (counts)

Access	Meaning	Where it is used
`context.read<T>()`	Get T once, without listening (used in callbacks)	Button handlers in `LikeButton`, `TripFormScreen._save`, `TripsScreen` swipe, `PlaceDetailsScreen._addToTrip`, `WeatherCard` (service)

The like button is a good example of why `select` matters. A list of 98 places has 98 like buttons. With `select`, liking one place rebuilds only the button whose answer changed, not the whole list:

lib/widgets/like_button.dart

```
Widget build(BuildContext context) {
  final isLiked = context.select<LikesProvider, bool>(
    (likes) => likes.isLiked(placeId),
  );
  return IconButton(
    tooltip: isLiked ? 'Unlike' : 'Like',
    icon: Icon(isLiked ? Icons.favorite : Icons.favorite_border),
    color: isLiked ? Colors.red : null,
    onPressed: () => context.read<LikesProvider>().toggle(placeId),
  );
}
```

The providers keep business rules out of the widgets. `TripsProvider` generates ids, keeps the list sorted, saves after every change and refuses duplicate stops:

lib/providers/trips_provider.dart — update, delete, undo and the save step shared by every change

```dart
Future<void> update(Trip trip) async {
  final i = _trips.indexWhere((t) => t.id == trip.id);
  if (i == -1) return;
  _trips[i] = trip;
  await _changed();
}

Future<void> delete(String id) async {
  _trips.removeWhere((t) => t.id == id);
  await _changed();
}

/// Puts a deleted trip back (used by the "Undo" snackbar).
Future<void> restore(Trip trip) async {
  if (byId(trip.id) != null) return;
  _trips.add(trip);
  await _changed();
}

/// Adds a place to an existing trip. Returns false if it was already there.
Future<bool> addPlace(String tripId, String placeId) async {
  final trip = byId(tripId);
  if (trip == null || trip.placeIds.contains(placeId)) return false;
  await update(trip.copyWith(placeIds: [...trip.placeIds, placeId]));
  return true;
}

Future<void> _changed() async {
  _sort();
  notifyListeners();
  await _store.saveTrips(_trips);
}

void _sort() => _trips.sort((a, b) => a.date.compareTo(b.date));
```

What would be harder without Provider

The first prototype of iLOcate (before this project) loaded the places in `main()` and passed the list through the constructors of `ILocateApp`, `HomeShell` and each tab, and kept likes in a global `ValueNotifier`. That worked for three screens but did not scale: the trip form, which is two levels below the trips tab, would have needed the place list and the trips list passed through screens that never use them, and the theme chosen in settings would have had to travel back up to `MaterialApp` through callbacks. The global notifier also could not be replaced in tests. With Provider, any screen can ask for exactly the state it uses, updates reach every listening screen automatically (liking a place in the detail screen updates the Likes tab and the badge at once), and tests build a `TripsProvider` with a mock store and check it directly.

7.3 API Integration

What the app needed

Two kinds of information cannot be bundled with the app because they change: the current weather at a place, and a road route between any two points. Both had to be fetched from a

server while the user waits, and the app had to stay usable if the request is slow or the phone has no signal, which is common on jeepneys.

How it was implemented

API	Endpoint	Model class	Used in
Open-Meteo Forecast API	`GET https://api.open-meteo.com/v1/forecast` ?latitude, longitude, current=temperature_2m, apparent_temperature, relative_humidity_2m, weather_code, wind_speed_10m & timezone=Asia/Manila	`Weather` (models/weather.dart)	`WeatherService` → `WeatherCard` on place details
OSRM Route Service	`GET https://router.project-osrm.org/route/v1/driving/{lng},{lat};{lng},{lat}` ?overview=full & geometries=polyline6	`DirectionsResult` (models/directions_result.dart)	`DirectionsService` → `DirectionsScreen`

Both are free and need no API key, so no secret is stored in the APK. The code is split into three layers so that UI code never touches HTTP or JSON: a shared `ApiClient`, one service class per API, and model classes with `fromJson` factories.

Layer 1: ApiClient. Every request goes through `ApiClient.getJson` (`lib/services/api_client.dart`). It adds a 12-second timeout and turns every kind of failure into an `ApiException` with a message the user can understand. The `isOffline` flag lets the UI show a no-Wi-Fi icon for connection problems.

lib/services/api_client.dart

```
Future<Map<String, dynamic>> getJson(Uri uri) async {
  final http.Response response;
  try {
    response = await _client.get(uri).timeout(timeout);
  } on TimeoutException {
    throw const ApiException(
      'The server took too long to answer. Please try again.',
      isOffline: true,
    );
  } on http.ClientException {
    // http wraps SocketException (no network, DNS failure) in ClientException.
    throw const ApiException(
      'No internet connection. Check your data or Wi-Fi and try again.',
      isOffline: true,
    );
  }

  if (response.statusCode ≠ 200) {
    throw ApiException('Server error (${response.statusCode}).');
  }
  try {
    return jsonDecode(response.body) as Map<String, dynamic>;
  } on FormatException {
    throw const ApiException('The server sent data we could not read.');
  }
}
```

Layer 2: services. `WeatherService` builds the query and parses the result. `DirectionsService` does the same for OSRM; note that OSRM expects longitude before latitude.

lib/services/weather_service.dart

```
Future<Weather> fetchCurrent(LatLng point) async {
  final uri = Uri.https(_host, '/v1/forecast', {
    'latitude': point.latitude.toStringAsFixed(4),
    'longitude': point.longitude.toStringAsFixed(4),
    'current': 'temperature_2m,apparent_temperature,relative_humidity_2m,'
        'weather_code,wind_speed_10m',
    'timezone': 'Asia/Manila',
  });
  final json = await _api.getJson(uri);
  try {
    return Weather.fromJson(json);
  } on Object {
    // Missing or mistyped fields: treat like any other bad response.
    throw const ApiException('Weather data was incomplete.');
  }
}
```

Layer 3: models. Sample responses captured from both APIs during testing (trimmed):

Open-Meteo response


```
{
 "latitude": 10.720562,
 "longitude": 122.51767,
 "timezone": "Asia/Manila",
 "current": {
  "time": "2026-10-27T23:00",
  "interval": 900,
  "temperature_2m": 25.0,
  "apparent_temperature": 28.7,
  "relative_humidity_2m": 91,
  "weather_code": 1,
  "wind_speed_10m": 14.5
 }
}
```


OSRM response


```
{
 "code": "Ok",
 "routes": [
  {
   "distance": 5513.6,
   "duration": 583.5,
   "geometry":
"u|hmSkcrwhFsO~PyG`GuHxIsYrZ_DlDaArAaNlPeX
v\\gDrCw...",
   "legs": [
    "..."
   ],
   "weight": 587.5
  }
 ],
 "waypoints": [
  "..."
 ]
}
```


lib/models/weather.dart

```dart
  factory Weather.fromJson(Map<String, dynamic> json) {
    final current = json['current'] as Map<String, dynamic>?;
    if (current == null) {
      throw const FormatException('Missing "current" block');
    }
    return Weather(
      temperature: (current['temperature_2m'] as num).toDouble(),
      feelsLike: (current['apparent_temperature'] as num).toDouble(),
      humidity: (current['relative_humidity_2m'] as num).toInt(),
      windSpeed: (current['wind_speed_10m'] as num).toDouble(),
      code: (current['weather_code'] as num).toInt(),
      time: DateTime.parse(current['time'] as String),
    );
  }
```

lib/models/directions_result.dart — the route geometry is decoded with the same polyline decoder used for jeepney routes

```dart
  factory DirectionsResult.fromJson(Map<String, dynamic> json) {
    if (json['code'] != 'Ok') {
      throw FormatException('OSRM returned ${json['code']}');
    }
    final routes = json['routes'] as List;
    if (routes.isEmpty) throw const FormatException('No route found');
    final best = routes.first as Map<String, dynamic>;
    return DirectionsResult(
      distanceMeters: (best['distance'] as num).toDouble(),
      durationSeconds: (best['duration'] as num).toDouble(),
      points: decodePolyline(best['geometry'] as String),
    );
  }
```

Loading, success and error states in the UI

`WeatherCard` (`lib/widgets/weather_card.dart`) uses a `FutureBuilder`. The future is created once in `initState`, not in `build`, so scrolling or liking the place does not send a new request. Retry simply creates a new future inside `setState`.

lib/widgets/weather_card.dart

```dart
late Future<Weather> _future;

@override
void initState() {
  super.initState();
  // Created once here, not in build(), so rebuilds don't refetch.
  _future = _fetch();
}

Future<Weather> _fetch() =>
    context.read<WeatherService>().fetchCurrent(widget.point);

void _retry() => setState(() => _future = _fetch());

@override
Widget build(BuildContext context) {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: FutureBuilder<Weather>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Row(children: [
              SizedBox.square(
                dimension: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              ),
              SizedBox(width: 16),
              Text('Checking the weather...'),
            ]);
          }
          if (snapshot.hasError) {
            return ErrorView(
              error: snapshot.error!,
              onRetry: _retry,
              compact: true,
            );
          }
          return _WeatherDetails(weather: snapshot.data!);
```

Situation	What the user sees	Figure
Request in progress	Spinner and "Checking the weather..." / spinner on the directions screen	—
Success	Temperature, condition, humidity, wind; route line, distance, time and jeepneys	Figures 7, 10

Situation	What the user sees	Figure
No internet / DNS failure	Wi-Fi-off icon, "No internet connection. Check your data or Wi-Fi and try again.", Retry	Figures 9, 12
Timeout (12 s)	"The server took too long to answer.", Retry	—
HTTP error (e.g. 500)	"Server error (500).", Retry	—
Bad or incomplete JSON	"Weather data was incomplete." / "Directions data was incomplete."	—
OSRM finds no road (code NoRoute)	"No road route found (...)", Retry	—

The directions screen also uses the API result together with local data. `jeepneysServing()` in `lib/services/commute_planner.dart` keeps only the routes that come within 500 m of both the start and the destination, and sorts them by total walking distance:

lib/services/commute_planner.dart

```
List<PujRoute> jeepneysServing(
  LatLng from,
  LatLng to,
  List<PujRoute> routes, {
  double walkMeters = 500,
}) {
  final matches = <(PujRoute, double)>[];
  for (final route in routes) {
    final walkFrom = _closestDistance(from, route.allPoints);
    if (walkFrom > walkMeters) continue;
    final walkTo = _closestDistance(to, route.allPoints);
    if (walkTo > walkMeters) continue;
    matches.add((route, walkFrom + walkTo));
  }
  matches.sort((a, b) => a.$2.compareTo(b.$2));
  return [for (final m in matches) m.$1];
}
```

What would break without this design

If the screens called `http.get` directly, each one would need its own timeout, status-code and exception handling, and it would be easy to forget a case and leave the user on an endless spinner. A missing `try` around a socket error would crash the screen when the phone loses signal. Without model classes the UI would read values like `json['current']['temperature_2m']` in widget code, so a renamed field would only be discovered at run time. Because the HTTP client is injected, the tests in `test/services_test.dart` replace it with a `MockClient` and check the success path, a 500 response, a network failure and an OSRM "NoRoute" answer without touching the internet.

7.4 CRUD and Local Storage

The trip is the entity the app fully manages. **Create:** the New trip button or "New trip with this place" opens `TripFormScreen`. **Read:** `TripsScreen` lists trips and `TripDetailsScreen` shows one.

Update: the edit button reopens the form with the trip's values, and Add to trip appends a stop. **Delete:** swipe on the list (with Undo) or the delete button on the details screen (with a confirmation dialog). All of these call `TripsProvider`, which saves through `LocalStore` (`lib/services/local_store.dart`), the only class that knows the `shared_preferences` keys:

lib/services/local_store.dart

```
  List<Trip> loadTrips() {
    final raw = _prefs.getString(_tripsKey);
    if (raw == null) return [];
    try {
      final list = jsonDecode(raw) as List;
      return list.map((e) => Trip.fromJson(e as Map<String, dynamic>)).toList();
    } on Object {
      // Corrupt data should not crash the app on launch; start fresh instead.
      return [];
    }
  }

  Future<void> saveTrips(List<Trip> trips) => _prefs.setString(
        _tripsKey,
        jsonEncode(trips.map((t) => t.toJson()).toList()),
      );
```

The form validates every field before saving: the name must be 3 to 40 characters, a date must be picked, notes are limited to 300 characters, and at least one place must be chosen. The date and places are not text fields, so they use custom `FormField` widgets that take part in the same `Form.validate()` call (Figure 16).

lib/screens/trip_form_screen.dart

```
String? validateTripTitle(String? value) {
  final text = (value ?? '').trim();
  if (text.isEmpty) return 'Give the trip a name';
  if (text.length < 3) return 'Name must be at least 3 characters';
  if (text.length > 40) return 'Name must be 40 characters or less';
  return null;
}
```

7.5 Error and Edge-Case Handling

Case	How it is handled	Where
No internet / timeout / server error	`ApiException` with `isOffline`; `ErrorView` with Retry	api_client.dart, error_view.dart
Bundled data fails to load	Error screen with Retry instead of the tabs	places_provider.dart, home_shell.dart
Empty lists	`EmptyState` with a message and, where useful, an action button	Places, Likes, Trips, Trip details
Invalid form input	`Form` validators and custom `FormField`s	trip_form_screen.dart
GPS off, permission denied or blocked	Specific message in a SnackBar; start point resets to the previous choice	location_service.dart, directions_screen.dart

Case	How it is handled	Where
Corrupt saved trips	`loadTrips()` catches the error and starts with an empty list	local_store.dart
Missing place photo	`PlaceImage` shows an icon placeholder instead of an error	place_image.dart
Trip deleted while its screen is open	Details screen shows "Trip not found"	trip_details_screen.dart
Unknown route or wrong argument type	`NotFoundScreen`	app_router.dart
Deleting by mistake	Undo in the SnackBar after a swipe; confirmation dialog on the details screen	trips_screen.dart, trip_details_screen.dart

7.6 Widget Composition and Code Organization

The code is split by responsibility. UI code in `screens/` and `widgets/` never imports `http` or `shared_preferences`; it only talks to providers and services.

```
lib/
  main.dart            providers and services registered with MultiProvider
  app.dart             MaterialApp, light/dark theme, named routes
  models/              Place, PujRoute, Trip, Weather, DirectionsResult
  services/            ApiClient, WeatherService, DirectionsService, LocationService,
                       LocalStore, AssetDataService, commute_planner
  providers/           PlacesProvider, LikesProvider, TripsProvider,
                       SettingsProvider, LocationProvider
  routes/app_router.dart
  screens/             home_shell, map, places, likes, trips, place_details,
                       directions, trip_details, trip_form, settings, not_found
  widgets/             reusable widgets (table below)
  theme/app_theme.dart
  utils/polyline.dart  polyline decoder shared by PUJ routes and OSRM
test/                  7 test files, 22 tests
```

Reusable widget	Kind	Used on
`PlaceTile`	Stateless	Places, Likes, Trip details
`PlaceImage`	Stateless	Map preview, Place details, PlaceTile, TripCard, Trip form, Place picker
`LikeButton`	Stateless (Provider-driven)	PlaceTile (all lists), Place details
`EmptyState`	Stateless	Places, Likes, Trips, Trip details, Not found
`ErrorView`	Stateless	Home loading error, WeatherCard, Directions
`OsmTileLayer` / `placeMarker`	Stateless / helper	Map, Place details, Directions
`WeatherCard`	Stateful (holds its Future)	Place details

Reusable widget	Kind	Used on
`TripCard`, `TypeChip`, place picker sheet	Stateless / Stateful	Trips list, Place details, Trip form

Stateful widgets are used only where a screen owns short-lived UI state: `HomeShell` (selected tab), `MapScreen` (visible routes, map controller), `PlacesScreen` (search text and filter), `DirectionsScreen` (chosen start and destination, request future), `TripFormScreen` (form fields) and `WeatherCard`. Everything shared across screens is in a provider. One theme built in `app_theme.dart` from the logo's teal color gives the whole app consistent colors in light and dark mode.

8. Software Engineering Tools and Practices

8.1 Git, GitHub and branching

The project is version-controlled with Git and hosted publicly on GitHub. Each feature was developed on its own branch, and every commit message describes one change. Each branch builds on the one before it, and its last commit passes `flutter analyze`, so each branch can be merged into `main` through a pull request without breaking the build. Commits reference the issue they address, for example *"Save liked places on the device (#1)"*.

Branch	Commits	What it adds	Issues
`feature/app-structure`	6	models/services/providers folders, PlacesProvider, saved likes, reusable widgets, named routes	#1
`feature/weather-api`	4	http package, ApiClient, Weather model and service, weather card	#4
`feature/gps-location`	3	geolocator, permissions, LocationProvider, my-location button	#5
`feature/directions`	4	OSRM model and service, jeepney matching, directions screen	#2, #3
`feature/trips-crud`	5	Trip model, TripsProvider, trips tab, form, details, add to trip	—
`feature/settings-theme`	2	saved theme mode, settings screen	—
`feature/release-prep`	4	launcher icon, splash screen, release signing config	—
`docs/readme`	2+	screenshots, README, this paper	—

> **Before submission:** push each branch and open its pull request into `main` in the order above (merging each before opening the next), with review comments on each, and add screenshots of the pull request pages here. The pull requests had not been opened when this paper was generated.

8.2 Issue tracking

Work was planned as GitHub Issues before coding started. Each issue describes one requirement, and the commit that resolves it mentions the issue number.

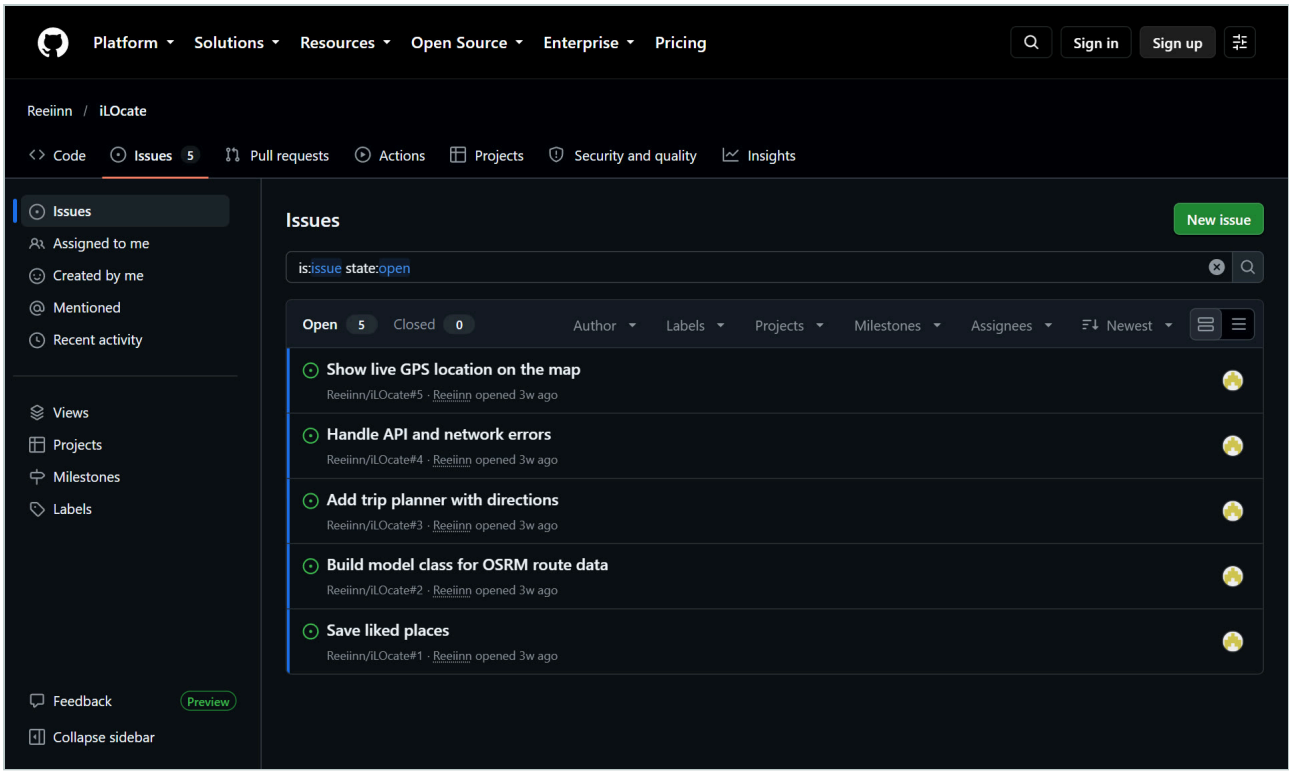


Figure 25. GitHub Issues for the repository, captured before the feature branches were merged.

Issue	Resolved by
#1 Save liked places	`LikesProvider` + `LocalStore` (shared_preferences)
#2 Build model class for OSRM route data	`DirectionsResult.fromJson` + tests in `api_models_test.dart`
#3 Add trip planner with directions	`DirectionsScreen`, `DirectionsService`, `jeepneysServing()`
#4 Handle API and network errors	`ApiClient`, `ApiException`, `ErrorView`, Retry
#5 Show live GPS location on the map	`LocationService`, `LocationProvider`, map button, Android permissions

8.3 Static analysis and testing

The project uses the `flutter_lints` rule set and is formatted with `dart format`. The final analysis run reports no issues:

Terminal

```
$ flutter analyze
Analyzing ilocate...
No issues found! (ran in 2.8s)

$ flutter test
00:04 +22: All tests passed!
```

Test file	What it checks
`polyline_test.dart`	Decoding an encoded polyline6 string

Test file	What it checks
`api_models_test.dart`	`Weather.fromJson` and `DirectionsResult.fromJson`, including bad responses
`services_test.dart`	Services with a `MockClient`: success, HTTP 500, network failure, OSRM NoRoute, lng/lat order
`commute_planner_test.dart`	Jeepney matching: near routes found and sorted, one-sided routes ignored
`models_test.dart`	`Place.fromJson`, `Trip` JSON round trip and `copyWith`
`providers_test.dart`	Trip create/read/update/delete, sorting, persistence across restart, duplicate stops; likes persistence
`trip_form_test.dart`	Title validator; widget test that an empty form shows all three errors and saves nothing

8.4 Debugging case: the icon generator would not compile

The bug. While preparing the release, running `dart run flutter_launcher_icons` failed before it generated any icon, and `dart run flutter_native_splash:create` failed the same way:

Terminal output

```
Failed to build flutter_launcher_icons:flutter_launcher_icons:
.../image-4.11.0/lib/src/formats/png_decoder.dart:377:39: Error: The method
'decodeLazy' isn't defined for the type 'ZLibDecoder'.
 - 'ZLibDecoder' is from 'package:archive/src/codecs/zlib_decoder.dart'
   ('.../archive-4.3.0/lib/src/codecs/zlib_decoder.dart').
    final input = const ZLibDecoder().decodeLazy(InputMemoryStream(imageData));
                                      ^^^^^^^^^^
```

How it was found. Reading the trace from the top showed that the failing file was not in the project: it was `png_decoder.dart` inside the `image` package, version 4.11.0. That file calls `ZLibDecoder.decodeLazy`. The second path in the message shows where `ZLibDecoder` came from: the `archive` package, version 4.3.0, which does not have that method. So the two packages in `pubspec.lock` were incompatible with each other. Neither is listed in `pubspec.yaml`; both are pulled in by the icon and splash generators. Running `flutter pub outdated` showed newer versions of both (`image` 4.12.0 and `archive` 4.5.2).

The fix. `flutter pub upgrade image archive` moved both packages to versions that work together. Both generators then ran successfully and produced the icon and splash files. The fix lives in the committed `pubspec.lock`, so anyone who clones the repository gets the working versions.

What I learned. A compile error can come from a dependency rather than from my own code. The file paths in the trace, including the package versions inside them, pointed straight to the cause. It was faster to read them than to search for the error message online.

> The rubric asks for a screenshot for this case. Add a screenshot of the terminal error or of Flutter DevTools from your own machine here.

8.5 Release preparation

Item	How it was done
App name	`android:label="iLOcate"` in `AndroidManifest.xml`; `title: 'iLOcate'` in `MaterialApp`
App icon	`flutter_launcher_icons` from `assets/images/logo.png`, including an adaptive icon (Figure 24)
Splash screen	`flutter_native_splash`: logo on white, dark background in dark mode, Android 12 splash API
Version	`1.0.0+2` in `pubspec.yaml`
Permissions	`INTERNET`, `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`
Signing	`build.gradle.kts` reads `android/key.properties` if present; otherwise it signs with the debug key

pubspec.yaml

```yaml
# App icon: dart run flutter_launcher_icons
flutter_launcher_icons:
  android: true
  ios: false
  image_path: assets/images/logo.png
  adaptive_icon_background: "#FFFFFF"
  adaptive_icon_foreground: assets/images/logo.png
  web:
    generate: true
    image_path: assets/images/logo.png
    background_color: "#FFFFFF"
    theme_color: "#0B8080"

# Splash screen: dart run flutter_native_splash:create
flutter_native_splash:
  color: "#FFFFFF"
  image: assets/images/logo.png
  color_dark: "#101414"
  android_12:
    color: "#FFFFFF"
    image: assets/images/logo.png
    color_dark: "#101414"
  ios: false
  web: true
```

Debug vs. release. A debug APK (`flutter build apk --debug`) includes the Dart VM in JIT mode, debugging and hot reload support, and assertion checks. It is large and runs slower, and is only meant for development. A release APK (`flutter build apk --release`) is compiled ahead of time to native ARM code, removes debugging support, shrinks unused icons from the font, and is the version that is installed by users and submitted.

```
# 1. get packages and check the code
flutter pub get
flutter analyze
flutter test

# 2. (re)generate icon and splash if the logo changed
dart run flutter_launcher_icons
dart run flutter_native_splash:create

# 3. build
flutter build apk --debug      # build/app/outputs/flutter-apk/app-debug.apk
flutter build apk --release    # build/app/outputs/flutter-apk/app-release.apk
```

> The release APK must be built on a machine with the Android SDK installed (Android Studio). It was not built while this paper was generated. Build it, install it on a phone, and add its file size and a photo of the installed icon here.

9. References

Packages (pub.dev)

Package	Version	Purpose	URL
provider	6.1.5+1	State management	https://pub.dev/packages/provider
http	1.6.0	REST requests	https://pub.dev/packages/http
shared_preferences	2.5.6	Local storage	https://pub.dev/packages/shared_preferences
flutter_map	8.3.2	Map widget	https://pub.dev/packages/flutter_map
latlong2	0.9.1	Coordinates, distance	https://pub.dev/packages/latlong2
geolocator	14.1.2	GPS position, permissions	https://pub.dev/packages/geolocator
intl	0.20.3	Date formatting	https://pub.dev/packages/intl
flutter_launcher_icons	0.14.4	App icon generation	https://pub.dev/packages/flutter_launcher_icons
flutter_native_splash	2.4.8	Splash screen generation	https://pub.dev/packages/flutter_native_splash
flutter_lints	5.0.0	Lint rules	https://pub.dev/packages/flutter_lints

APIs and data

1. Open-Meteo. *Weather Forecast API*. https://open-meteo.com/en/docs (data licensed CC BY 4.0).
2. Project OSRM. *OSRM API Documentation, Route Service*. https://project-osrm.org/docs/v5.24.0/api/
3. OpenStreetMap contributors. *Map tiles and data*, ODbL. https://www.openstreetmap.org/copyright
4. Google. *Encoded Polyline Algorithm Format*. https://developers.google.com/maps/documentation/utilities/polylinealgorithm
5. WMO. *Weather interpretation codes* (as listed in the Open-Meteo documentation).

6. Place photos and jeepney route data: the author's earlier iLOcate web application.

Documentation and tutorials

1. Flutter. *Navigate with named routes* and *Send data to a new screen*. https://docs.flutter.dev/cookbook/navigation
2. Flutter. *Simple app state management* (Provider). https://docs.flutter.dev/data-and-backend/state-mgmt/simple
3. Flutter. *Fetch data from the internet*. https://docs.flutter.dev/cookbook/networking/fetch-data
4. Flutter. *Build a form with validation*. https://docs.flutter.dev/cookbook/forms/validation
5. Flutter. *Build and release an Android app*. https://docs.flutter.dev/deployment/android
6. Flutter. *Mock dependencies using Mockito / http MockClient*. https://docs.flutter.dev/cookbook/testing/unit/mocking
7. flutter_map documentation. https://docs.fleaflet.dev

AI tools

1. Anthropic. *Claude Code* (model Claude Opus 5.5), October 2026. Used as a coding assistant for restructuring the prototype, writing parts of the implementation and tests, generating screenshots, and drafting this documentation. All code was reviewed, run and tested by the author, who can explain every part of it.